OKOUME LABS
SAS · START-UP GABONAISE · AGRITECH

Fermia

CHALLENGE AGRI TRACK 241 · PROGRAMME POUFA · LIVRABLE N° 4

Documentation technique Identifiants, synchronisation, API et formats d'échange

Règles de construction des identifiants, règle de gestion des conflits de synchronisation, API et formats d'échange de la plateforme Fermia.

8
familles de codes automatiques, jamais réutilisés

1 règle
dernière saisie horodatée retenue, conflit signalé

JSON
API REST documentée en OpenAPI

CSV · XLSX
reprise et exports, QR signé

À l'attention de :
Centre Gabonais de l'Innovation (CGI)
Direction Générale de l'Élevage (DGE), Programme POUFA

Réf. **OKL-POUFA-2026-DOC-004**
Libreville, 9 novembre 2026
Version 1.0, document confidentiel

Documentation technique

Livrable n° 4 du cahier des charges Agri Track 241 : règles de construction des identifiants, règle de gestion des conflits de synchronisation, API et formats d'échange de la plateforme Fermia (programme POUFA, Gabon).

Sommaire

1 Règles de construction des identifiants 3

2 Règle de gestion des conflits de synchronisation 6

3 API : accès, conventions, sécurité 12

4 API : principaux groupes d'endpoints 15

5 Formats d'échange : import, exports, QR, EMA-i Plus 18

6 Points à valider et écarts constatés 21

Comment lire ce document

Ce document décrit ce que le code de la plateforme fait aujourd'hui (branche principale du dépôt, état au 9 novembre 2026). Les chemins de fichiers cités sont ceux du dépôt, sous `backend/src/` sauf mention contraire. Quand un élément n'existe pas dans le code, il est marqué **Proposé / à valider** : il ne fait pas partie de ce qui est livré.

Repère	Sens
Existant	Fonction présente dans le code et testée.
Proposé / à valider	Proposition de l'éditeur, absente du code, à confirmer avec la DGE et le CGI avant tout développement.
`FCFA`	Tous les montants sont des entiers en francs CFA (pas de décimales).
`Africa/Libreville`	Fuseau métier : UTC+1 toute l'année, sans heure d'été.

Vue d'ensemble

Serveur NestJS (Node.js) et base PostgreSQL, un seul programme par instance (POUFA) ; application web (pilotage, administration, saisie, file hors ligne dans le navigateur) ; application Android (saisie terrain hors connexion) ; page publique de lot consultée par QR code, sans compte.

Émetteur : Okoume Labs SAS, Libreville, le 9 novembre 2026. Réf. OKL-POUFA-2026-DOC-004.

1 Règles de construction des identifiants

Les identifiants du programme sont automatiques, uniques, jamais modifiés ni réutilisés (CDC §7.5 et §9.4). Ils se lisent à l'œil nu : le code d'un lot dit de quelle bande, de quelle ferme et de quelle province il vient.

1.1 Principe : un compteur atomique qui ne recule jamais

Tous les numéros de la hiérarchie ferme, bande, lot, sous-lot viennent d'une table de compteurs (`id_counters`, service `programme/id-counter.service.ts`). Le numéro suivant est pris par un upsert atomique `ON CONFLICT … next + 1 RETURNING`, dans la transaction de l'appelant. Conséquences :

- deux saisies simultanées ne peuvent pas obtenir le même numéro ;
- une bande abandonnée ne rend pas son numéro : le compteur n'est jamais décrémenté ;
- un numéro s'élargit au-delà de son remplissage au lieu de déborder (`G1-10000`, `B100`, `L100`) ;
- le code d'une ferme et son programme sont immuables (déclencheur SQL `farms_programme_code_immutable`).

1.2 Hiérarchie des codes

Objet	Format	Exemple	Compteur (`scope`)
Ferme	province + `-` + n° sur 4 chiffres	`G1-0012`	`FARM:<province>`
Localité	province + `-` + n° sur 2 chiffres	`G1-06`	`LOCALITE:<provinceId>`
Bande	code ferme + `-B` + rang sur 2 chiffres	`G1-0012-B07`	`BANDE:<code ferme>`
Livraison d'intrants	code ferme + `-L` + rang sur 2 chiffres	`G1-0012-L03`	`LIVRAISON:<code ferme>`
Lot de traçabilité (racine)	code bande + `-L` + rang sur 2 chiffres	`G1-0012-B07-L01`	`LOT:<code bande>`
Sous-lot (un par cession)	code lot + `-` + rang sur 2 chiffres	`G1-0012-B07-L01-03`	`SOUS_LOT:<code lot>`
Opérateur de la filière	`OP-` + n° sur 4 chiffres	`OP-0001`	`OPERATEUR:<programme>`
Alerte sanitaire officielle	`AS-` + année + `-` + n° sur 4 chiffres	`AS-2026-0001`	`ALERTE_SANITAIRE:<année>`

Générateurs : `programme/codes.ts` (`farmCode`, `localiteCode`, `bandeCode`), `supply/supply-rules.ts` (`deliveryCode`), `trace/trace-codes.ts` (`lotCode`, `subLotCode`, `operatorCode`), `surveillance/sanitary-alert-rules.ts` (`sanitaryAlertCode`). La bande est ouverte par `batches/poufa/bande-opening.service.ts`.

1.3 Lecture d'un code

```
G1 – 0012 – B07 – L01 – 03
|    |      |     |     └ 3e cession (sous-lot) du lot L01
|    |      |     └──────── 1er lot de traçabilité de la bande
|    |      └──────────────── 7e bande ouverte sur la ferme (jamais réutilisée)
|    └─────────────────────── 12e ferme de la province
└──────────────────────────── province G1 (G1 à G9 : les 9 provinces semées,
                              programme/data/gabon-provinces.ts)
```

Garantie. Un code, une fois remis (étiquette imprimée, bon de livraison, QR), reste valable et désigne toujours le même objet. Aucune réutilisation n'est possible, même après abandon.

1.4 Identifiants techniques

Les clés internes (utilisateurs, événements, saisies, ventes, commandes…) sont des UUID générés par PostgreSQL (`PrimaryGeneratedColumn('uuid')`). Ils servent aux URL de l'API (`:farmId` , `:batchId` …) mais ne sont jamais affichés comme identifiant de programme ; seuls les codes ci-dessus le sont. Une valeur mal formée dans une URL (UUID, date) répond 400 « Paramètre invalide », jamais 500.

1.5 Références documentaires (ventes, commandes, abattage, comptabilité)

Les pièces commerciales et comptables portent une référence lisible construite autrement que les codes de la hiérarchie : préfixe, date du jour à Libreville (`AAAAMMJJ`), puis un tirage aléatoire à 6 chiffres. L'unicité est vérifiée avant l'enregistrement (jusqu'à 5 tirages).

Pièce	Format	Exemple	Unicité et source
Vente	`VTE-AAAAMMJJ-######`	`VTE-20261108-482913`	Contrôle applicatif ; clé d'idempotence unique par ferme (`finance/sales.service.ts`).
Commande	`CMD-AAAAMMJJ-######`	`CMD-20261108-105577`	`orders/orders.service.ts` (`nextReference`).
Ordre d'abattage	`ABT-AAAAMMJJ-######`	`ABT-20261108-730241`	`slaughter/slaughter.service.ts` ; repris dans le bordereau.
Code interne d'abattage	`ABT-AAAAMMJJ-######-I`	`ABT-20261108-730241-I`	Suivi interne à l'envoi ; tirage sous verrou transactionnel (`pg_advisory_xact_lock`) ; l'opérateur peut saisir le sien.
Écriture comptable	`EC-AAAAMMJJ-######`	`EC-20261108-318604`	Date = date de l'écriture ; en cas de collision concurrente, nouvel essai dans un point de sauvegarde (`accounting/accounting.service.ts`).

Pièce	Format	Exemple	Unicité et source
Client de la ferme	`CL-####`	`CL-0007`	Maximum existant + 1, par ferme (`finance/customers.service.ts`).

> **Remarque.** Ces références ne passent pas par la table de compteurs : elles sont uniques mais non consécutives, et ne remplacent pas les codes de traçabilité. Le document source `backend/docs/identifiants.md` ne les décrit pas encore (voir section 6). Le numéro d'écriture n'est donc pas une numérotation continue au sens d'un journal légal ; une numérotation séquentielle par exercice est **Proposé / à valider** .

1.6 Adresse et signature du QR code d'un lot

Le QR code d'un lot ne contient pas un texte libre mais une adresse signée :

```
<TRACE_PUBLIC_BASE_URL>/l/<code du lot>?s=<signature>
exemple : https://<domaine public>/l/G1-0012-B07-L01?s=Qm3x9Tb2LkP7vWc1
```

- **Signature** : HMAC-SHA256 du texte `fermia:trace-lot:v1:<code>` avec la clé serveur `TRACE_QR_SECRET` , encodé en base64url et tronqué à 16 caractères (96 bits). Elle est calculée une seule fois, à la création du lot, et stockée sur le lot (`qrToken`).
- **Étiquettes déjà imprimées** : changer la clé n'invalide aucune étiquette, car un scan est comparé à la valeur stockée (comparaison en temps constant). Seuls les lots créés ensuite utilisent la nouvelle clé.
- **Pourquoi signer** : les codes se suivent ; sans signature, n'importe qui pourrait deviner le lot suivant ou imprimer un faux QR.
- **Configuration** : `TRACE_QR_SECRET` et `TRACE_PUBLIC_BASE_URL` sont obligatoires en production (`trace/trace-config.ts`). Les valeurs ne figurent pas dans ce document.
- **Page publique** : `GET /public/lots/:code?s=<signature>` . La forme du code est contrôlée avant toute requête en base (`^[A-Z0-9][A-Z0-9-]{2,59}$`) ; un QR inconnu ou mal signé reçoit toujours la même réponse 404, sans indice sur l'existence du code. Chaque scan est journalisé (adresse IP pseudonymisée par HMAC, zone de 0,1° au plus).

1.7 Règles de conservation

Règle	Application
Jamais modifié	Code ferme et programme (déclencheur SQL) ; codes de bande, lot, sous-lot (aucune route de modification).
Jamais réutilisé	Compteurs croissants uniquement, y compris après abandon d'une bande.
Idempotence à la reprise	La clé de reprise (référence AMI, ou code ferme et numéro de bande) est conservée : relancer un fichier ne crée aucun doublon (section 5).

2 Règle de gestion des conflits de synchronisation

Règle du cahier des charges : **la dernière saisie horodatée est retenue, et le conflit est signalé**. Aucune donnée n'est perdue : la valeur écartée est conservée et consultable.

La règle s'applique à toutes les fermes (l'instance ne sert que le programme POUFA). Elle est codée dans `sync/conflict-rules.ts` (règle pure, testée), enregistrée par `sync/sync.service.ts` et appelée par la saisie journalière (`daily-entries/daily-entries.service.ts`).

2.1 La file d'attente de l'appareil

Dans les deux applications, toute écriture passe par une fonction unique « envoyer ou mettre en file » : `sendOrQueue` (`web/src/offline/engine.ts` ; `mobile/src/offline/engine.ts`). Le principe est le même sur le web et sur Android :

- **Écrire d'abord sur l'appareil.** L'opération est inscrite dans la file locale *avant* la requête (navigateur : `localStorage`, partagé entre onglets ; mobile : stockage local de l'application). Fermer l'application entre la requête et l'accusé de réception ne perd rien : la reprise réutilise la même clé.
- **Envoyer si le réseau répond** ; sinon l'opération reste en file et part au retour du réseau.
- **Une opération** = identifiant unique entre appareils, type (`OfflineKind`), ferme, méthode, chemin, corps, libellé en français, heure de création, compte qui l'a saisie (jamais rejouée sous la session d'un autre utilisateur), compteur d'erreurs serveur.
- **Un seul passage à la fois** : un verrou (`navigator.locks` sur le web) évite que deux onglets envoient la même file.

2.2 Types d'écritures hors ligne (`OfflineKind`)

Type	Plateformes	Création à clé d'idempotence
`daily-entry` (saisie journalière)	web, mobile	Non : mise à jour par jour, règle de conflit (2.8)
`sale` , `feed-input` , `stock-loss`	web, mobile	Oui
`order-create` , `order-payment` , `customer-payment`	web, mobile	Oui
`order-deliver` , `order-cancel`	web, mobile	Transitions : un 2e envoi est refusé, abandonné sans effet
`pdv-create` , `pdv-update` , `pdv-delete` (points de vente)	web, mobile	Création : oui
`stock-transfer-create` , `stock-transfer-cancel`	web, mobile	Création : oui
`health-event` , `treatment` , `prophylaxis-complete`	web, mobile	Oui (rejouer rend l'enregistrement existant)

Type	Plateformes	Création à clé d'idempotence
`delivery-confirm` (réception d'intrants)	web, mobile	Oui (réponse `replay: true`)
`band-sortie` (web) / `sortie` (mobile)	web, mobile	Oui (unique par ferme et clé)
`visit` (web) / `farm-visit` (mobile)	web, mobile	Oui ; photos non mises en file sur le web
`farm-gps`	mobile	Positionnement GPS de la ferme

2.3 Clé d'idempotence

Pour une création, l'identifiant de l'opération fait office de clé d'idempotence (`idempotencyKey`), à la tentative en direct comme à chaque reprise. Le serveur impose l'unicité du couple (ferme, clé) : un nouvel envoi de la même opération renvoie l'enregistrement déjà créé, sans doublon, même si l'accusé de réception s'est perdu en route.

```
POST /farms/{farmId}/bandes/{batchId}/sorties   (corps)
{ "idempotencyKey": "off-sortie-lq3k2x-9f3a1c…",   // identifiant de l'opération
  "capturedAt": "2026-11-08T08:40:12+01:00",         // fixé au 1er essai
  "deviceSentAt": "2026-11-08T11:00:03+01:00",       // renouvelé à chaque envoi
  … }
```

2.4 Horodatage de capture, ordre d'envoi, rejet

Sujet	Règle
Horodatage de capture	`capturedAt` est l'heure de l'appareil quand l'utilisateur enregistre. Elle est fixée une fois pour toutes à la création de l'opération (format ISO avec décalage `+01:00` sur mobile) : une reprise garde la même heure. Le champ `deviceSentAt` (heure de l'appareil à l'envoi) est lui renouvelé à *chaque* tentative.
Ordre d'envoi	Premier entré, premier sorti, par ferme. Tant que des opérations plus anciennes de la ferme attendent, une nouvelle opération ne les double pas : la file est d'abord vidée, sinon la nouvelle se range derrière. Ainsi une saisie ancienne ne peut pas écraser une correction plus récente, et un encaissement n'arrive pas après la livraison.
Erreur serveur (5xx) ou réseau	L'opération reste en file et bloque la suite de la file. Après 10 erreurs serveur d'affilée (`MAX_SERVER_ATTEMPTS`), elle cesse de bloquer : refusée avec le message « Le serveur échoue à répétition sur cette opération » (sauf pour une opération d'argent, qui reste en file et est simplement passée).
Rejet définitif (4xx)	Une donnée refusée par le serveur (400, 403, 404, 409…) quitte la file mais n'est **jamais perdue en silence** : elle est rangée dans la liste des opérations rejetées, avec le message du serveur, pour être ressaisie. Le 401 (session expirée) garde l'opération en file : elle repart après la prochaine connexion.

Sujet	Règle
Argent	Les encaissements ne sont jamais abandonnés : si la caisse a été clôturée depuis l'encaissement, l'opération reste en file « en attente de caisse » (`heldByClosedCaisse`). La caisse n'est jamais rouverte à zéro automatiquement.

Écart de comportement entre applications : sur le mobile, les erreurs 408, 425 et 429 sont rejouées ; sur le web, toute erreur 4xx autre que 401 est un rejet. Sur le mobile, un rejet concerne les créations et les opérations d'argent ; une simple transition (livrer, annuler, supprimer) est abandonnée car le refus signifie en général qu'elle a déjà eu lieu.

2.5 Règle de conflit par type d'écriture

Type d'écriture	Règle
Saisie journalière (mise à jour d'une fiche par jour)	Comparaison **champ par champ**. La saisie dont l'heure de capture corrigée est la plus récente gagne ; à égalité, la valeur du serveur reste. Le conflit est enregistré, la valeur perdante conservée, le propriétaire et les agents notifiés (2.8 et 2.9).
Créations à clé (vente, sortie, traitement, réception…)	Pas de conflit de valeur : l'idempotence évite le doublon. Un rejeu rend l'enregistrement existant.
Sortie d'animaux	Création idempotente ; l'heure de capture est corrigée selon la même règle d'horloge.
Livraison d'une commande	Transition : une seule fois. Le 2e envoi est refusé (« déjà livrée ») et abandonné. Le jour de livraison est celui de la capture (2.10).
Annulation, mise à jour, suppression	Transitions : si l'état a déjà changé, le serveur refuse (4xx) et l'opération est écartée ou rangée parmi les rejets.
Écriture en ligne (sans `capturedAt` ni `baseUpdatedAt`)	S'applique directement, sans contrôle de conflit ; le serveur note quand même qui a écrit chaque champ et quand.

2.6 Ce que l'appareil envoie pour la saisie journalière

Champ	Sens
`capturedAt`	Heure de la saisie sur l'appareil.
`baseUpdatedAt`	Version (`updatedAt`) de la fiche telle que l'appareil la connaissait. Absent si la fiche est créée hors ligne.
`baseValues`	Valeurs connues avant modification (morts, aliment en kg, eau, poids moyen, nombre pesé, œufs).
`deviceSentAt`	Heure de l'appareil à l'envoi, pour mesurer l'écart d'horloge.
`deviceId`	Identifiant aléatoire de l'installation (téléphone) ou du navigateur.

```
POST /farms/{farmId}/batches/{batchId}/daily-entries
{ "entryDate": "2026-11-05",
  "deaths": 7, "waterL": 30,
  "capturedAt":    "2026-11-05T08:40:00+01:00",
  "baseUpdatedAt": "2026-11-05T07:10:22.000Z",
  "baseValues":    { "deaths": 4, "waterL": null },
  "deviceSentAt":  "2026-11-05T11:00:00+01:00",
  "deviceId":      "d3c1…" }
```

2.7 Horloge de l'appareil

1. À chaque envoi portant `deviceSentAt`, le serveur calcule l'écart *heure appareil moins heure serveur* et l'enregistre (`sync_clock_samples` : utilisateur, ferme, appareil, écart en millisecondes).
2. Si l'écart dépasse le paramètre `SYNC_ECART_HORLOGE_MAX_S` (300 secondes par défaut, plage 0 à 86 400, modifiable par l'administrateur, versionné), l'heure de saisie est **corrigée de l'écart** avant toute comparaison, et la correction est marquée (`clockCorrected`).
3. Une heure de saisie n'est jamais dans le futur : elle est ramenée à l'heure du serveur.

Un téléphone mal réglé ne gagne donc pas tous les conflits.

2.8 Comparaison champ par champ

La saisie journalière n'écrase que les champs envoyés, et le mobile renvoie des zéros pour les champs qu'il ne modifie pas. Pour chaque champ :

#	Cas	Résultat
1	Champ non modifié par l'appareil (valeur envoyée égale à `baseValues`)	Non écrit ; la valeur du serveur reste.
2	Pas de modification concurrente (inchangé sur le serveur depuis `baseUpdatedAt` , ou dernier écrit par le même appareil)	La valeur envoyée s'applique.
3	Même valeur des deux côtés	S'applique, sans conflit.
4	Conflit : changé sur le serveur depuis la version connue, par un autre appareil, avec une autre valeur	**La capture la plus récente (heure corrigée) gagne** ; à égalité, la valeur du serveur reste.

Une fiche créée hors ligne alors qu'une fiche existe déjà pour le même jour est traitée comme une modification concurrente de chaque champ déjà saisi. Champs comparés : `deaths` , `feedQuantity` (kg), `waterL` , `avgWeightKg` , `sampleSize` , `eggsCollected` , `eggsSellable` , `eggsCracked` , `eggsSmall` , `eggsDoubleYolk` , `eggsDirty` . Les autres champs (phase et type d'aliment, unité, source) suivent l'écriture.

2.9 Aucune donnée perdue, et ce que voit l'utilisateur

Chaque conflit crée une ligne dans `sync_conflicts` : fiche, champ, gagnant (`SERVEUR` ou `APPAREIL`), **valeur du serveur et valeur reçue**, heures de capture (brute et corrigée), utilisateurs, appareils, écart d'horloge. Les tables sont couvertes par le journal d'audit. Lecture : `GET /farms/:farmId/sync-conflicts?batchId=` .

Où	Ce que voit l'utilisateur
Centre de synchronisation	Les opérations en attente (libellé en français, montant d'argent en attente) et les opérations **rejetées** avec le message exact du serveur, à ressaisir.
Fiche de bande (web : onglet de synthèse ; mobile : panneau)	Liste des conflits, du plus récent au plus ancien : « Saisie du 05/11, morts », **valeur retenue** et valeur écartée (conservée), motif (« la saisie la plus récente a gagné » avec les deux heures, ou « valeur du serveur conservée à égalité »), mention « heure corrigée » si l'horloge était déréglée. Les agents et profils institutionnels qui lisent la bande voient la même liste.
Notification	Après validation de l'écriture, événement `SYNC_CONFLICT`. Règles réglables (`/programme/notification-rules`) : `FERME_CONFLIT_SAISIE` (propriétaire et administrateurs de la ferme), `AGENT_CONFLIT_SAISIE` (agents techniques qui suivent la ferme). La notification ouvre la fiche de la bande.

2.10 Jour de référence : Africa/Libreville

Les jours métier sont comptés à l'heure de Libreville (UTC+1, sans heure d'été) : `gabonDay()` dans `orders/delivery-day.ts` et `todayLocal()` dans `common/utils/date.utils.ts`. Une livraison faite hors ligne le lundi et synchronisée le mercredi est donc **cédée le lundi** : la date de livraison, l'écriture de chiffre d'affaires et, pour un client lié à un opérateur, la date de cession (sous-lot et QR) suivent la capture corrigée, et non le jour de synchronisation. Sans `capturedAt` (écriture en ligne), c'est le jour du serveur. Une livraison entre 00 h 00 et 01 h 00 locales n'est pas datée de la veille.

2.11 Exemple complet

Heure réelle	Appareil	Action
08 h 10	Tablette de l'agent (en ligne)	Saisie du jour : 4 morts.
08 h 40	Téléphone de l'éleveur (hors ligne)	Même jour : 7 morts, 30 L d'eau.
11 h 00	Téléphone (retour du réseau)	Envoi avec `capturedAt` = 08 h 40.

Le champ « morts » a changé sur le serveur depuis un autre appareil : conflit. La capture du téléphone (08 h 40) est plus récente que celle de la tablette (08 h 10) : **7 morts est retenu**, la valeur 4 est conservée dans `sync_conflicts` et le conflit est signalé. Le champ « eau » n'avait pas été saisi par la tablette : 30 L s'applique sans conflit.

Si le téléphone avait eu deux heures d'avance (envoi daté de 13 h 00 pour 11 h 00 au serveur), sa capture aurait été ramenée de 10 h 40 à 08 h 40 avant la comparaison, et la saisie marquée « heure corrigée ».

```
{ "date": "2026-11-05", "field": "deaths", "winner": "APPAREIL",
  "serverValue": 4,  "incomingValue": 7,
  "serverCapturedAt":   "2026-11-05T08:10:00+01:00",
  "incomingCapturedAt": "2026-11-05T08:40:00+01:00",
  "clockCorrected": false }
```

Forme indicative d'un conflit tel que lu par `GET /farms/:farmId/sync-conflicts` ; les noms exacts des champs sont dans la documentation OpenAPI en ligne.

> **Évolutions proposées.** Proposé / à valider Une résolution manuelle d'un conflit par le propriétaire (« rétablir la valeur écartée ») n'existe pas : seule la correction par une nouvelle saisie ou, pour l'administrateur, `POST /farms/:farmId/bandes/:batchId/corrections`. De même, la détection de conflit sur d'autres écritures que la saisie journalière n'est pas implémentée : ces écritures sont des créations idempotentes ou des transitions à état unique.

3 API : accès, conventions, sécurité

L'API REST de Fermia est celle utilisée par l'application web et l'application Android ; il n'existe pas d'API distincte « partenaires ». Elle est décrite en OpenAPI (Swagger) et fonctionne en JSON sur HTTPS.

3.1 Accès

Élément	Valeur
Base de production	`https://fermia.greenleaves.ga/api` (le serveur d'entrée retire le préfixe `/api` avant de transmettre à l'API).
Documentation OpenAPI interactive	Swagger UI (`SwaggerModule` , `main.ts`) : titre « API Fermia, Système d'information POUFA », version 1.0, filtre et tri alphabétique, jeton conservé. Chemin configurable (`SWAGGER_PATH` , défaut `api-docs`) ; la configuration de déploiement publie `/api/docs/` . Désactivée en production sauf `SWAGGER_ENABLED=true` . **Proposé / à valider** confirmer l'ouverture de cette page au jury.
Format	JSON (UTF-8). Fichiers (PDF, CSV, XLSX) sur les routes d'export. Téléversements en `multipart/form-data` .
Corrélation	En-tête `X-Request-Id` renvoyé sur chaque réponse (identifiant du journal d'audit). Autres en-têtes exposés : `Content-Disposition` , `X-Audit-Truncated` , `X-Report-Fingerprint` , `X-Report-Sha256` .

3.2 Authentification

- **Connexion** : `POST /auth/login` avec le numéro de téléphone et le code secret du compte. Réponse : `accessToken` (JWT) et le profil `user` (`id` , `phone` , `fullName` , `role`).
- **Utilisation** : en-tête `Authorization: Bearer <accessToken>` sur toutes les routes, sauf les routes publiques (3.3).
- **Durée** : 7 jours par défaut (`JWT_EXPIRES_IN`) ; **8 heures** pour un compte institutionnel (champ `expiresIn` dans la réponse).
- **Comptes institutionnels** : second facteur par code SMS (`POST /auth/login/otp`) quand il est activé sur l'instance (`INSTITUTION_OTP=sms`) ; un jeton émis sans cette étape est refusé.
- **Révocation** : un jeton émis avant la dernière réinitialisation du code du compte est refusé (« Session expirée : le code a été changé, reconnectez-vous »), avec 1 s de tolérance. Un compte suspendu perd l'accès immédiatement, sans attendre l'expiration du jeton.
- **Limite de connexion** : 5 requêtes par minute sur `/auth/login` et `/auth/login/otp` , 10 par minute sur l'inscription acheteur, 30 par minute sur la page publique d'un lot, 120 par minute par défaut sur le reste. Dépassement : 429. Derrière un reverse proxy, `TRUST_PROXY` doit être renseigné pour que la limite s'applique par client.
- **Comptes** : seul l'acheteur peut s'inscrire (`POST /auth/register-buyer`). Les exploitants sont créés par l'administration du programme (`POST /programme/farms/:farmId/activate-owner`) ; agents et institutions par rattachement d'accès.

```
POST /auth/login
{ "phone": "+241XXXXXXXX", "code": "<code secret>" }

200 { "accessToken": "eyJ…", "user": { "id": "…", "role": "PROPRIETAIRE", … } }
```

3.3 Routes publiques (sans jeton)

`POST /auth/register-buyer`, `POST /auth/login`, `POST /auth/login/otp`, `GET /market/listings[/:id]` (catalogue du Market, réponse sur liste blanche : ni identifiant de bande, ni coût, ni GPS, ni téléphone) et `GET /public/lots/:code` (page de traçabilité).

3.4 Conventions

Sujet	Convention
Dates et heures	Jour calendaire : `AAAA-MM-JJ`. Instant : ISO 8601 (`2026-11-05T08:40:00+01:00` ou en UTC avec `Z`). Un jour `AAAA-MM-JJ` n'est jamais décalé par fuseau ; les jours métier sont ceux de Libreville.
Montants	Entiers en FCFA (`…Fcfa`), jamais de décimales. Poids en kg, volumes en litres.
Pagination	Paramètres facultatifs `?limit=&offset=` sur les listes (`limit` entier positif, `offset` entier positif ou nul, maximum 500 par page). Sans eux, la liste est rendue entière. Valeur invalide : 400.
Validation	Les corps sont validés (champs inconnus ignorés). Les messages d'erreur sont en français et destinés à être affichés tels quels.
Rejeu	Les créations hors ligne portent `idempotencyKey` (section 2.3).

3.5 Codes de réponse et format d'erreur

Code	Sens
200 / 201	Succès (lecture, modification / création).
400	Donnée invalide : champ manquant, valeur hors bornes, identifiant ou date mal formé (« Paramètre invalide », « Date invalide »), règle métier non respectée.
401	Pas de jeton, jeton expiré ou révoqué, compte suspendu, identifiants incorrects.
403	Authentifié mais non autorisé : rôle, permission de ferme, droit de la matrice, ou ferme hors du périmètre du compte.
404	Ressource inconnue (ou hors périmètre). Un QR de lot inconnu ou mal signé répond toujours un 404 identique.
409	Conflit d'état : par exemple une opération déjà effectuée ou une ressource déjà existante.
429	Limite de requêtes dépassée.

```
{ "statusCode": 400,
  "message": "Le numéro de téléphone est obligatoire.",
  "error": "Bad Request" }
```

Format standard NestJS : `message` est un texte, ou une liste de textes pour une erreur de validation à plusieurs champs. Toute requête refusée (401, 403, 404, 429, corps invalide) est elle aussi inscrite au journal d'audit (`audit/audit-exception.filter.ts`).

3.6 Droits : rôles, permissions de ferme et matrice du programme

Chaque route traverse, dans cet ordre : authentification JWT, accès institutionnel (`ProgramAccessGuard`), rôle (`RolesGuard`), permission de ferme (`PermissionsGuard`).

Rôle	Portée
`PLATFORM_ADMIN`	Administration du programme (registre, listes, paramètres, reprise, journal d'audit). Tous les droits.
`PROPRIETAIRE` , `ELEVEUR`	Compte d'une ferme. Le propriétaire passe toutes les permissions de sa ferme ; un éleveur ou agent de ferme doit détenir la permission (`saisie:creer` , `production:gerer` , `vente:creer` , `stock:gerer` , `compta:rapports` …), réglée par ferme.
`ACHETEUR`	Market uniquement ; aucun accès aux données d'une ferme.
`INSTITUTION`	DGE, services vétérinaires, BCEG, MAEDR, ANINF, agents techniques. Ses droits viennent d'un **profil d'accès** (matrice domaine × action) et d'un **périmètre** (national, province, fermes) portés par son rattachement. Chaque route porte un domaine (`@Domain` : TECHNIQUE, REGISTRE, PILOTAGE, TRACABILITE, AUDIT…). Refus par défaut. Les fonctions bonus de Fermia sont en lecture seule pour une institution, quelle que soit la matrice.
`OPERATEUR`	Réservé (abattoirs, distributeurs) : aucune route ouverte pour l'instant.

La matrice est **paramétrable par l'administration** sans développement ; les profils par défaut s'appliquent à l'initialisation (la DGE valide le registre, par exemple).

3.7 Journal d'audit des lectures institutionnelles

Toutes les écritures sont journalisées. Les **lectures sensibles** le sont aussi (`@AuditRead` , `@AuditThirdPartyRead`) lorsque le lecteur est un tiers de la ferme : compte institutionnel ou administrateur de la plateforme, avec le motif en clair. Les lectures par le propriétaire et l'équipe de sa propre ferme ne sont pas des consultations par un tiers et ne sont pas journalisées. Les exports de fichiers (`@AuditExport`) sont enregistrés comme `EXPORT` . Le journal est consultable par l'administration (`GET /audit/events` , export `/audit/events/export` , scellés `/audit/seals` , vérification `POST /audit/seals/verify`).

4 API : principaux groupes d'endpoints

Sélection des routes les plus importantes par module. La liste exhaustive, avec les schémas de corps et de réponse, est dans la **documentation OpenAPI en ligne** (3.1). `PROP` = propriétaire, `ELEV` = éleveur ou agent de ferme (avec permission), `ADMIN` = administrateur de la plateforme, `INST` = institution selon la matrice (domaine indiqué). Les chemins sont relatifs à la base ; `:farmId` et `:batchId` sont des UUID.

Authentification et ferme

Méthode	Chemin	Rôle / droit	Objet
POST	`/auth/login`	public (5/min)	Connexion, jeton JWT
POST	`/auth/login/otp`	public (5/min)	Second facteur SMS (institutions)
POST	`/auth/register-buyer`	public	Inscription d'un acheteur
GET	`/farms`	PROP, ELEV, ACHETEUR	Fermes accessibles au compte
PATCH	`/farms/:farmId`	PROP, ELEV · reglages:ferme	Réglages de la ferme
GET	`/farms/:farmId/permissions`	PROP, ELEV · equipe:gerer	Permissions effectives de l'équipe

Registre et administration du programme (domaine REGISTRE)

Méthode	Chemin	Rôle / droit	Objet
GET	`/programme/farms`	ADMIN	Registre des fermes
POST	`/programme/farms`	ADMIN	Créer une ferme (code G1-0012 attribué)
POST	`/programme/farms/:farmId/status`	ADMIN	Changer le statut (historique conservé)
PUT	`/programme/farms/:farmId/gps`	ADMIN, PROP, ELEV · registre:gps	Positionner la ferme (historique GPS)
POST	`/programme/farms/:farmId/activate-owner`	ADMIN	Activer le compte de l'exploitant
GET	`/programme/parameters`	ADMIN, PROP, ELEV	Paramètres du programme (versionnés)
POST	`/programme/parameters/:key`	ADMIN	Modifier un paramètre (ex. SYNC_ECART_HORLOGE_MAX_S)

Bandes, saisies et synchronisation (domaine TECHNIQUE)

Méthode	Chemin	Rôle / droit	Objet
POST	`/farms/:farmId/batches`	PROP, ELEV · production:gerer	Ouvrir une bande (code G1-0012-B07)
GET	`/farms/:farmId/bandes/:batchId`	PROP, ELEV · INST	Fiche de bande, indicateurs, historique

Méthode	Chemin	Rôle / droit	Objet
POST	`/farms/:farmId/batches/:batchId/daily-entries`	PROP, ELEV · saisie:creer	Saisie journalière (champs de conflit inclus)
GET	`/farms/:farmId/batches/:batchId/daily-entries`	PROP, ELEV	Lister les saisies du lot
POST	`/farms/:farmId/bandes/:batchId/sorties`	PROP, ELEV · saisie:creer	Sortie d'animaux (idempotente)
POST	`/farms/:farmId/batches/:batchId/cloture`	PROP, ELEV · production:gerer	Clôturer la bande
GET	`/farms/:farmId/sync-conflicts`	PROP, ELEV · INST	Conflits de synchronisation (?batchId=)

Traçabilité et abattage (domaine TRACABILITE)

Méthode	Chemin	Rôle / droit	Objet
GET	`/farms/:farmId/trace/lots`	PROP, ELEV	Lots de traçabilité de la ferme
POST	`/farms/:farmId/trace/lots/:lotId/validate`	ADMIN, PROP, ELEV	Valider un lot
GET	`/farms/:farmId/trace/lots/:lotId/label`	PROP, ELEV	Étiquette imprimable avec QR
GET	`/trace/lots/by-code/:code`	ADMIN, PROP, ELEV	Retrouver un lot par son code
GET	`/trace/operators`	ADMIN, PROP, ELEV	Opérateurs de la filière (OP-0001)
POST	`/trace/operators`	ADMIN	Créer un opérateur
GET	`/public/lots/:code?s=`	public (30/min)	Page publique du lot (QR)
POST	`/farms/:farmId/slaughter-orders`	PROP, ELEV · production:abattage	Ordre d'abattage (ABT-…)
POST	`/farms/:farmId/slaughter-orders/:orderId/process`	PROP, ELEV · production:abattage	Traiter l'abattage
GET	`/farms/:farmId/batches/:batchId/passeport`	PROP, ELEV	Passeport de la bande

Ventes, commandes et Market

Méthode	Chemin	Rôle / droit	Objet
POST	`/farms/:farmId/sales`	PROP, ELEV · vente:creer	Vente (VTE-…), idempotente
POST	`/farms/:farmId/orders`	PROP, ELEV · vente:commande	Commande client (CMD-…)
POST	`/farms/:farmId/orders/:orderId/livrer`	PROP, ELEV · vente:commande	Livrer ; jour = capture corrigée
GET	`/market/listings`	public	Catalogue public (liste blanche)
POST	`/market/orders`	ACHETEUR	Commande d'un acheteur
POST	`/farms/:farmId/market/listings`	PROP, ELEV (droit market)	Publier une offre
POST	`/farms/:farmId/market/orders/:id/accept`	PROP, ELEV (droit market)	Accepter une commande Market

Finance et comptabilité (bonus, ouverts en lecture aux institutions)

Méthode	Chemin	Rôle / droit	Objet
POST	`/farms/:farmId/caisse/open`	PROP, ELEV · caisse:ouvrir	Ouvrir la caisse
POST	`/farms/:farmId/expenses`	PROP, ELEV · compta:depense	Enregistrer une dépense
GET	`/farms/:farmId/rentabilite/overview`	PROP, ELEV · compta:rapports	Rentabilité (export associé)
POST	`/farms/:farmId/accounting/entries`	PROP, ELEV · compta:ecritures	Écriture comptable (EC-…)
GET	`/farms/:farmId/accounting/journal`	PROP, ELEV · compta:lire	Journal ; aussi grand-livre, balance, bilan
GET	`/farms/:farmId/accounting/export?format=csv|pdf`	PROP, ELEV · compta:rapports	Export comptable (SYSCOHADA)

Santé, intrants et surveillance

Méthode	Chemin	Rôle / droit	Objet
GET	`/farms/:farmId/input-deliveries`	PROP, ELEV · intrants:lire	Livraisons d'intrants reçues
POST	`/farms/:farmId/input-deliveries/:id/confirm`	PROP, ELEV · stock:gerer	Confirmer la réception
POST	`/programme/input-deliveries`	ADMIN	Enregistrer une livraison (G1-0012-L03)
GET	`/surveillance/alerts`	INST (surveillance)	Alertes sanitaires officielles (AS-…)
POST	`/surveillance/alerts/:alertId/publish`	INST (surveillance)	Publier une alerte
GET	`/surveillance/declarations`	INST (surveillance)	Déclarations sanitaires des fermes

Pilotage, reprise et audit

Méthode	Chemin	Rôle / droit	Objet
GET	`/pilotage/dashboard`	INST (pilotage)	Tableau de bord national, par province
GET	`/pilotage/exports/farms.csv | .xlsx`	INST (pilotage)	Export du pilotage (audité)
POST	`/programme/imports`	ADMIN (registre)	Déposer un fichier de reprise
POST	`/programme/imports/:id/integrate`	ADMIN (registre)	Intégrer la reprise
GET	`/audit/events`	ADMIN (audit)	Journal d'audit

5 Formats d'échange

5.1 Import de reprise : CSV et Excel

Module `data-import`, procédure complète dans `backend/docs/reprise-des-donnees.md`. Toutes les routes relèvent du domaine Registre, avec portée nationale. Droits de la matrice : consulter, créer (déposer, essai, intégrer), modifier (colonnes, corrections, rejet), valider (par défaut la DGE).

Élément	Spécification
Types de reprise	`AMI` (une ligne par ferme ; seuls `nom` et `province` obligatoires), `BANDES_HISTORIQUES` (une ligne par saisie journalière ; `ref_bande`, ferme, `date_mise_en_place`, `effectif_initial` obligatoires), `OUTIL_TRANSITOIRE` (identifiant de bande de l'outil).
Modèles	`GET /programme/imports/templates/:type` : en-tête et ligne d'exemple (`modele-reprise-ami.csv`, `-bandes.csv`, `-outil-transitoire.csv`).
Fichier	CSV (séparateur `,` ou `;`, UTF-8 ou Latin-1) et classeur Excel `.xlsx` / `.xlsm` (première feuille). Les anciens formats `.xls` et `.ods` sont refusés avec un message clair. Maximum 5 000 lignes et 5 Mo par reprise.
Dates et nombres	Dates `jj/mm/aaaa` ou `aaaa-mm-jj` ; décimales avec virgule ou point. Les noms de colonnes courants (« Nom de la ferme », « N° bande », « Mortalité ») sont reconnus ; une colonne inconnue est ignorée et signalée.
Empreinte	SHA-256 du fichier conservé avec les lignes dans une zone de transit.

Circuit contrôlé et rejouable

Étape	Route	Effet
Dépôt et contrôle	`POST /programme/imports` (`file`, `type`, `note`)	Rien n'est écrit au registre ; contrôles : doublons, référentiels, effectifs, dates. États : Analysé, Contrôlé.
Correspondance	`PUT …/:id/settings`	Associer les colonnes si elles ne sont pas reconnues.
Correction	`PATCH …/:id/rows/:line`	Corriger une valeur ou exclure une ligne (motif d'au moins 5 caractères, conservé).
Essai à blanc	`POST …/:id/dry-run`	Exécute et annule ; état Essai ; rapport d'essai.
Validation	`POST …/:id/validate`	Droit Registre : Valider ; seulement après un essai.
Intégration	`POST …/:id/integrate`	Une seule transaction ; une bande est intégrée entièrement ou pas. Clé de reprise conservée : aucun doublon au rejeu.
Rapport	`GET …/:id/report`, `/report/pdf`, `/report/csv`	JSON, PDF à signer, CSV ligne à ligne ; empreinte SHA-256 dans l'en-tête `X-Report-Fingerprint`.

> **Intégration d'une ferme déjà au registre.** Seuls les champs vides sont complétés ; une valeur différente n'est jamais écrasée, elle est signalée. Les sorties et ventes des bandes historiques ne sont pas reprises.

5.2 Exports

Export	Formats	Route et remarques
Pilotage : provinces, fermes, bandes	CSV, XLSX	`GET /pilotage/exports/{provinces|farms|bandes}.{csv|xlsx}` ; agrégats ; petits groupes masqués pour un profil agrégé ; export audité.
Rapports de pilotage	PDF	`/pilotage/reports/runs/:id/pdf` ; modèles configurables par l'administration.
Comptabilité	PDF (défaut), CSV	`GET /farms/:farmId/accounting/export?format=csv` ; journal, grand-livre, balance, bilan (plan SYSCOHADA).
Rentabilité	fichier	`…/rentabilite/overview/export` , `…/batches/:batchId/export` .
Ventes, commandes	PDF	Reçu `…/sales/:saleId/receipt` , bon de commande `…/orders/:orderId/bon-de-commande` , reçu de commande.
Abattage	PDF	Bordereau `…/slaughter-orders/:orderId/bordereau` .
Traçabilité	fichier	Étiquette de lot avec QR `…/trace/lots/:lotId/label` .
Journal d'audit	fichier	`GET /audit/events/export` ; en-tête `X-Audit-Truncated` si l'export est tronqué ; scellés vérifiables.
Rapport de reprise	JSON, PDF, CSV	Voir 5.1 ; empreinte SHA-256.

Les fichiers sont nommés par l'en-tête `Content-Disposition` . Les montants restent des entiers en FCFA et les dates au format `AAAA-MM-JJ` . Les en-têtes d'empreinte (`X-Report-Fingerprint` , `X-Report-Sha256`) permettent de vérifier qu'un document signé correspond aux données.

5.3 QR de traçabilité : contenu et page publique

Le QR contient l'adresse signée décrite en 1.6. La page publique répond en JSON, sur liste blanche : jamais de téléphone, de point GPS précis, de coût, de nom de personne ni d'identifiant interne.

```
GET /public/lots/G1-0012-B07-L01?s=<signature>      →  200
{ "code": "G1-0012-B07-L01", "kind": "…", "status": "…",
  "quantity": 120, "cededOn": "2026-11-08",
  "product":  { "species": "POULET", "speciesLabel": "Poulet",
                "type": "CHAIR", "typeLabel": "Chair" },
  "producer": { "farmName": "…", "localite": "…", "province": "…" },
  "band":     { "code": "G1-0012-B07", "openedOn": "2026-09-14",
                "closedOn": null },
  "operatorType": "…",
  "consommonsGabonais": { "label": "Consommons Gabonais", "url": "…" },
  "notice": "Ce QR code indique l'origine du produit. Il ne constitue
             pas un certificat sanitaire." }
```

- La mention « pas un certificat sanitaire » est obligatoire (CDC §6.2 et §8).
- Un lot clos reste affiché (statut CLOS) : ses sujets continuent d'être vendus après la clôture de la bande.
- Un QR inconnu ou mal signé répond 404, avec le message « QR code inconnu ou invalide », sans cache ; une page valide est mise en cache **privé** 5 minutes seulement, pour ne pas masquer les scans au journal.
- Paramètres facultatifs `lat` et `lng` : position du visiteur, avec son accord ; seule une zone de 0,1° (environ 11 km) est conservée.

5.4 Échange avec EMA-i Plus (FAO)

Absent du code. L'échange avec EMA-i Plus (exigence 5.2 du cahier des charges) n'est **pas implémenté** : il figure comme « protocole à formaliser » dans le suivi de conformité (`docs/cdc-traceability.md` , test en attente `it.todo`), et aucun module du serveur ne l'appelle. **Proposé / à valider**

Proposition de cadrage, à valider avec la DGE, le service vétérinaire et la FAO :

- Sens : **sortant** (Fermia vers EMA-i Plus) pour les alertes sanitaires officielles et déclarations de suspicion, **entrant** facultatif pour les foyers publiés. Les alertes publiées disposent déjà d'une source interne (`reporting/integration-sources.ts` , alertes `AS-AAAA-####`).
- Format : JSON sur HTTPS, authentification par clé d'API propre à l'intégration, codes ferme et province comme clés de rapprochement, journalisation de chaque échange au journal d'audit.
- Reprise sur erreur : file d'envoi avec clé d'idempotence, selon le principe de la section 2.3.
- Contrainte : aucune donnée personnelle ni coordonnée GPS précise dans les échanges.

6 Points à valider et écarts constatés

Cette section liste ce qui n'existe pas dans le code (marqué **Proposé / à valider**) et les écarts relevés entre la documentation du dépôt (`backend/docs/*.md`) et le code, à la date du document. Les écarts sont à corriger dans les fichiers sources.

6.1 Points à valider (absents du code)

#	Point	État
1	Échange EMA-i Plus (FAO) : protocole, sens, format, authentification.	Non implémenté ; protocole à fournir.
2	Ouverture de la documentation Swagger au jury : publique, restreinte ou sur demande.	Chemin `/api/docs/` prévu par la configuration de déploiement ; ouverture à confirmer.
3	Numérotation continue des écritures comptables par exercice (aujourd'hui date + tirage).	Proposition, non implémentée.
4	Résolution manuelle d'un conflit de saisie (rétablir la valeur écartée).	Absente ; correction par nouvelle saisie ou par l'administrateur.
5	Conflit champ par champ sur d'autres écritures que la saisie journalière.	Non nécessaire à ce jour (créations idempotentes, transitions à état unique).
6	Accès « partenaires » à l'API (clés d'API par institution, hors comptes utilisateurs).	Absent : accès par compte et matrice.
7	Comptes opérateurs (abattoirs, distributeurs).	Rôle réservé ; aucune route ouverte.

6.2 Écarts entre la documentation du dépôt et le code

#	Document	Écart constaté
A	`poufa-conflits-hors-ligne.md` §6	Indique que les jours de livraison sont « comptés en UTC ». Le code (`orders/delivery-day.ts`, `gabonDay`) lit le jour à l'heure de Libreville (UTC+1). La présente documentation retient le comportement du code.
B	`reprise-des-donnees.md` §1 et « Ce qui n'est pas repris »	Affirme que les classeurs Excel `.xlsx` ne sont pas encore lus. Le code les lit (`data-import/import-file-readers.ts`, première feuille d'un `.xlsx` ou `.xlsm`, test inclus). Le CSV reste accepté.
C	`identifiants.md`	Ne couvre pas les références de pièces `VTE-`, `CMD-`, `ABT-` (et code interne `-I`), `EC-` et `CL-`, qui existent dans le code et sont aléatoires ou à maximum + 1, non tirées de la table de compteurs. À compléter. Le préfixe `EC-` désigne les écritures comptables ; les ventes portent `VTE-`.

#	Document	Écart constaté
D	Applications web et mobile	Comportements légèrement différents face aux 4xx (408, 425, 429 rejoués sur le mobile, rejetés sur le web) et noms de types différents (`band-sortie` / `sortie` , `visit` / `farm-visit`). Le moteur web est un « port » du moteur mobile, à tenir synchronisé.

6.3 Confidentialité de ce document

Ce document ne contient aucun secret : ni clé de signature (`JWT_SECRET` , `TRACE_QR_SECRET`), ni mot de passe, ni code de démonstration. Les exemples de jetons, de signatures et d'identifiants sont fictifs.

> **Contact.** Okoume Labs SAS, Libreville. Document établi le 9 novembre 2026 à partir de l'état de la branche principale du dépôt de la plateforme Fermia.